EXPERIMENT NO. 9

Binary Search

Programming Language: Java

Name : A Yuvan Charan
Register No. : 192424383

Aim

To write a Java program that checks whether a given key exists in an array using binary search and to analyse its time complexity.

Question

Check if a given number x exists in an array using binary search. Analyze its time complexity using Big-O notation.

Test Case 1

Input: X = {3, 4, 6, -9, 10, 8, 9, 30}, KEY = 10

Output: Element 10 is found at position 5

Test Case 2

Input: X = {3, 4, 6, -9, 10, 8, 9, 30}, KEY = 100

Output: Element 100 is not found

Note: Binary search needs sorted data. The array is therefore sorted together with the original positions of its elements, so the position reported is the position in the original array.

Algorithm

Step 1: Start.

Step 2: Store each element with its original position and sort the items by value.

Step 3: Set low = 0 and high = n - 1.

Step 4: While low <= high, find mid = (low + high) / 2.

Step 5: If the value at mid equals the key, print its original position and stop; if it is smaller than the key, set low = mid + 1; otherwise set high = mid - 1.

Step 6: If the loop ends without a match, print that the element is not found.

Step 7: Stop.

Program (Java)

```
public class Main {

    // items[i][0] = value, items[i][1] = original position (1-based)
    static void sortItems(int[][] a) {              // insertion sort by value
        for (int i = 1; i < a.length; i++) {
            int[] key = a[i];
```

```
            int j = i - 1;
            while (j >= 0 && a[j][0] > key[0]) {
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
    }

    static void binarySearch(int[] x, int key) {
        int n = x.length;
        int[][] items = new int[n][2];
        for (int i = 0; i < n; i++) {
            items[i][0] = x[i];
            items[i][1] = i + 1;
        }
        sortItems(items);

        int low = 0, high = n - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (items[mid][0] == key) {
                System.out.println("Element " + key + " is found at position " +
items[mid][1]);
                return;
            }
            if (items[mid][0] < key)
                low = mid + 1;
            else
                high = mid - 1;
        }
        System.out.println("Element " + key + " is not found");
    }

    public static void main(String[] args) {
        int[] x = {3, 4, 6, -9, 10, 8, 9, 30};

        binarySearch(x, 10);
        binarySearch(x, 100);
    }
}
```

Output

```
Element 10 is found at position 5
Element 100 is not found

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: O(log n) for the search itself, because the search range is halved at every step (best case O(1)). The one-time sort costs O(n^2) here with insertion sort (O(n log n) with a better sort); on an already sorted array only the O(log n) search is needed.

Space Complexity: O(1) for the search; O(n) for the array of items with original positions.

Result

Thus the program was executed successfully and the output was verified.